ChitChatBusiness backend — deploy from the AWS Console + GitHub

This is the **no-CLI** path. You click through the AWS Console and connect it to your GitHub repo so that every push to the branch rebuilds and redeploys the backend automatically.

- **Repo:** `https://github.com/Edhost-Pty-Ltd/ChitChatBusiness.git`
- **Branch:** `ChitchatBusiness_postgress`
- **Backend path in repo:** `server/` (Dockerfile is at `server/Dockerfile`)
- **Service used: AWS App Runner** — the one AWS service whose console connects straight to GitHub and auto-deploys on push.

Why App Runner (and two honest caveats)

App Runner is the right fit because it: builds from your Dockerfile, has a built-in **GitHub connection UI**, terminates HTTPS for you, supports WebSockets (so Socket.IO works), and auto-scales — no load balancer or cluster to wire up by hand.

Two things App Runner will **not** do for you, handled later in this guide:

1. **Database migrations** are not run automatically. You run them once from a small separate step (Phase 7).
2. App Runner has **no separate worker process** — the BullMQ push/cleanup workers run inside the same service, which is fine for this app.

> If you later outgrow App Runner (e.g. you want a dedicated worker service or fine-grained Socket.IO sticky control), the ECS + ALB console path is the next step. For now App Runner is the simplest GitHub-connected deploy.

Phase 0 — Prerequisites

- An AWS account with Console access.
- The backend code pushed to GitHub on branch `ChitchatBusiness_postgress`, **including** `server/Dockerfile`. (Confirm the Dockerfile is visible in the repo on github.com before continuing.)
- Your Firebase service-account JSON contents (from `server/firebase-service-account.json`) ready to paste.
- Pick one Region (top-right of the Console) and stay in it for everything.

Phase 1 — Create the data stores first (Console)

App Runner needs these to exist so you can paste their endpoints as env vars.

1a. RDS PostgreSQL

1. Console > **RDS** > **Create database**.
2. **Standard create**, Engine **PostgreSQL**, version **16.x**.
3. Templates: **Free tier** or **Dev/Test**.
4. Settings: DB instance id `chitchat-pg`, master username `chitchat`, set a strong master password (save it).
5. Instance: `db.t4g.micro` is fine to start. Storage 20 GB.
6. Connectivity: **Do not** make it publicly accessible. Note the VPC.
7. Additional config > **Initial database name:** `chitchat`.
8. Create. When status is **Available**, open it and copy the **Endpoint**.
9. Enforce TLS: Console > RDS > **Parameter groups** > create one for postgres16, set `rds.force_ssl = 1`, and attach it to the instance (Modify > apply). This makes TLS mandatory.

Your connection string: `postgres://chitchat:<password>@<rds-endpoint>:5432/chitchat`

1b. ElastiCache for Redis

1. Console > **ElastiCache** > **Redis OSS caches** > **Create**.
2. Cluster mode **disabled**, name `chitchat-redis`, node `cache.t4g.micro`, replicas 0 (to start).
3. **Encryption in transit: Enabled** (this is what lets the app use `rediss://`).
4. Same VPC as RDS. Create.
5. When available, copy the **Primary endpoint** > `rediss://<redis-endpoint>:6379`.

1c. S3 bucket

1. Console > **S3** > **Create bucket**, name e.g. `chitchat-media-prod`, same Region.
2. Keep **Block all public access ON** (the app uses presigned URLs).
3. Create.

1d. An IAM user with S3 keys (for the app)

The app reads `S3_ACCESS_KEY_ID` / `S3_SECRET_ACCESS_KEY`.

1. Console > **IAM** > **Users** > **Create user** `chitchat-s3`.
2. Attach an inline policy allowing `s3:PutObject`, `s3:GetObject`, `s3:DeleteObject` on `arn:aws:s3:::chitchat-media-prod/*`.
3. Create an **access key** (type: Application outside AWS) and save both values.

1e. Networking note (important)

App Runner connects to RDS/ElastiCache (which are private) through a **VPC connector**. You'll create that in Phase 4. Make sure RDS and ElastiCache security groups will allow inbound from the App Runner connector's security group — easiest is to pick/create one security group now and use it for the connector, then add inbound rules on RDS (5432) and Redis (6379) from that SG.

Phase 2 — Store secrets (optional but recommended)

So secrets aren't typed as plaintext env vars:

1. Console > **Secrets Manager** > **Store a new secret** > **Other type**.
2. Add key/value pairs: `DATABASE_URL`, `REDIS_URL`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `FIREBASE_SERVICE_ACCOUNT_JSON` (paste the whole JSON).
3. Name it `chitchat/backend`. Save. You'll reference these in Phase 5.

(If you'd rather move fast, you can paste them directly as env vars in Phase 5 instead — App Runner supports both.)

Phase 3 — Connect App Runner to GitHub (the key step)

1. Console > **App Runner** > **Create service**.
2. **Source:** choose **Source code repository**.
3. Click **Add new** next to Connection > this opens the **GitHub connection** flow:
   - Give the connection a name, e.g. `chitchat-github`.
   - Click **Install another account** (or use an existing install). You're redirected to GitHub to authorize the **AWS Connector for GitHub** app.
   - On GitHub, install it on the **Edhost-Pty-Ltd** organization and grant access to the **ChitChatBusiness** repository (you can scope it to just that repo). Approve.
   - Back in the Console, the connection shows **Available**.
4. **Repository:** select `Edhost-Pty-Ltd/ChitChatBusiness`.
5. **Branch:** select `ChitchatBusiness_postgress`.

6. **Source directory:** set to `server` (so App Runner builds from the backend folder, not the repo root).
7. **Deployment trigger:** choose **Automatic** — now every push to that branch redeploys.

Phase 4 — Build settings (use the Dockerfile)

1. On the next screen, **Configuration file / Build settings:** choose **Use a configuration file** is *not* required — instead pick **Dockerfile**-based build:
   - Build type: **Dockerfile**.
   - Dockerfile path: `Dockerfile` (relative to the `server` source directory you set in Phase 3).
   - Leave the build context as the source directory.

> App Runner builds the exact same multi-stage image your `server/Dockerfile` defines, so what runs in the cloud matches local.

Phase 5 — Service settings (port, env, secrets)

1. **Service name:** `chitchat-backend`.
2. **Virtual CPU & memory:** 1 vCPU / 2 GB is a safe start.
3. **Port: 4000** (the app listens on 4000).
4. **Environment variables** — add the non-secret ones directly:

Key	Value
NODE_ENV	production
PORT	4000
DATABASE_SSL	require
S3_REGION	your Region, e.g. us-east-1
S3_ENDPOINT	https://s3.<region>.amazonaws.com
S3_BUCKET	chitchat-media-prod
S3_FORCE_PATH_STYLE	false
CORS_ORIGINS	your app origin(s), e.g. https://app.yourdomain.com
FIREBASE_PROJECT_ID	chitchatbusiness-880ce
GEMINI_MODEL	gemini-3.8-flash (only if using AI minutes)

5. **Secrets** — for each sensitive value, add an env var and choose **Reference a Secrets Manager value**, pointing at the matching key in `chitchat/backend`: `DATABASE_URL`, `REDIS_URL`, `S3_ACCESS_KEY_ID`, `S3_SECRET_ACCESS_KEY`, `FIREBASE_SERVICE_ACCOUNT_JSON`.
   - If you skipped Secrets Manager, paste them as plain env vars here instead.
   - The App Runner **instance role** must be allowed to read that secret — the Console offers to create/choose this role; grant it `GetSecretValue`.
6. **Health check:** set the path to `/health` (protocol HTTP). App Runner marks the service healthy only when Postgres and Redis both respond.
7. **Networking > Outgoing traffic:** choose **Custom VPC** and **Add new VPC connector**:
   - Pick the **same VPC** as RDS/ElastiCache.
   - Select the private subnets and the security group from Phase 1e.
   - Save. This is what lets the service reach the private database and cache.
8. Review > **Create & deploy**.

Phase 6 — Watch the first deploy

1. App Runner shows build + deploy logs live. Wait for status **Running**.
2. It assigns a URL like `https://xxxx.<region>.awsapprunner.com` — this is your API base URL.
3. If it fails health checks, it's almost always: the VPC connector can't reach RDS/Redis (fix the security-group inbound rules), or a missing env var (the app logs the exact variable name and exits).

Phase 7 — Run database migrations (one time + on schema changes)

App Runner won't run migrations. Two console-friendly options:

Option A — temporary command override (simplest):

1. App Runner > your service > **Configuration** > **Edit**.
2. Temporarily change the **Start command** to: `node dist/db/migrate.js up`
3. Save & deploy. Watch the logs for `Migrations up complete`, then the task exits — App Runner will report unhealthy because the server didn't start. That's expected for this one run.
4. **Revert** the start command back to empty (the Dockerfile default `node dist/index.js`) and deploy again.

Option B — run migrations elsewhere (cleaner, repeatable):

- Temporarily make RDS publicly accessible (or use a bastion / CloudShell in the VPC), run `node dist/db/migrate.js up` once against `DATABASE_URL`, then lock RDS back down.

Migrations are idempotent (`node-pg-migrate` tracks applied ones), so re-running is safe. Do this whenever a deploy adds a new file under `server/migrations/`.

Phase 8 — Custom domain (optional)

1. App Runner > your service > **Custom domains** > **Link domain**.
2. Enter `api.yourdomain.com`. App Runner gives you DNS records (a validation CNAME + a target). Add them at your DNS provider / Route 53.
3. Once validated, your API is reachable at the custom domain over HTTPS.

Phase 9 — Verify it's live

1. In a browser (or any REST client), open: `https://<your-apprunner-url>/health` Expect: `{"status":"ok","dependencies":{"postgres":true,"redis":true}}`.
2. Sign in from the app with a real Firebase phone token > an authenticated call should succeed (not 401).
3. Send a message between two clients > arrives in real time (Socket.IO over the App Runner URL).
4. Upload media > presigned S3 upload succeeds.

Phase 10 — The ongoing workflow (what "connected to GitHub" buys you)

From now on:

1. You make backend changes locally.
2. `git push origin ChitchatBusiness_postgress`.
3. App Runner detects the push, rebuilds the Dockerfile image, and rolls out the new version automatically — zero clicks.
4. Only extra manual step: if the push added a **new migration file**, run Phase 7 once after the deploy.

Point the mobile app at the service by setting in the repo-root `.env`:

```
EXPO_PUBLIC_API_URL=https://<your-apprunner-url-or-custom-domain>
EXPO_PUBLIC_SOCKET_URL=https://<your-apprunner-url-or-custom-domain>
```

and make sure that origin is in the backend `CORS_ORIGINS`.

Troubleshooting (specific to this backend)

- **Service won't start, logs name a variable** > that env var/secret is missing. The server validates config on boot and exits naming it. Add it in Phase 5.
- `/health` **returns 503** > Postgres or Redis unreachable. Check the VPC connector + security-group inbound rules, `DATABASE_SSL=require`, and that `REDIS_URL` uses `rediss://` (encryption in transit is on).
- **401 on every request** > the `FIREBASE_SERVICE_ACCOUNT_JSON` secret is malformed; re-paste the full JSON.
- **Build fails** > confirm `server/Dockerfile` is pushed and the **Source directory** is `server` with Dockerfile path `Dockerfile`.
- **Media upload fails** > check the S3 IAM key and that `S3_FORCE_PATH_STYLE=false` with the AWS `S3_ENDPOINT`.